

RANKTIER: EFFICIENT MIXTURE-OF-EXPERTS PREFILL ON AHEAD-OF-TIME COMPILED INFERENCE ACCELERATORS

Anonymous Authors¹

ABSTRACT

Mixture-of-experts (MoE) language models choose each token’s experts at run time, but server inference accelerators such as Qualcomm Cloud AI 100 execute ahead-of-time (AOT) compiled programs with static shapes and no data-dependent control flow. The number of tokens each expert processes, its capacity, is therefore fixed at compile time, and because some expert in some layer receives nearly every token of a prefill chunk, a program that never drops a token pads every expert to the chunk length. The compiler’s lowering of the MoE combine adds a second cost that grows with the chunk: a dense accumulator reduced in serialized steps and rooted on one device. We present RankTier, which keeps the program static and moves the decisions it cannot compile into data. Each device ranks its experts by load at run time, binds fixed-capacity tiers to load ranks instead of expert identities, and fetches the selected weights through an index gather that the compiler lowers to indirect DMA; a token-owned combine then lets each device sum only its own tokens’ expert rows before one distributed cross-device sum. On Qwen3-30B-A3B across the four SoCs of a Cloud AI 100 Ultra card, with the KV cache on device, RankTier prefills 128-, 256- and 512-token chunks $4.05\times$, $2.45\times$ and $2.94\times$ faster than the production compilation, without dropping a token in any measured prompt. Against a baseline with the same compilation settings but the compiler’s combine and every expert padded to the chunk length, the reduction redesign alone is $2.06\text{--}2.38\times$ faster, while the rank-bound tiers pay only on top of it, adding $1.06\text{--}1.37\times$.

1 INTRODUCTION

Mixture-of-experts (MoE) layers let a language model grow its parameter count without a matching growth in per-token compute: a router sends each token to a few of many experts, and only those experts run (Shazeer et al., 2017; Fedus et al., 2022). Recent open models use MoE layers throughout; Qwen3-30B-A3B routes each token to 8 of 128 experts in each of its 48 layers and activates 3.3B of its 30.5B parameters (Yang et al., 2025), and Mixtral and DeepSeek-V3 follow the same pattern (Jiang et al., 2024; DeepSeek-AI, 2024). During prefill, a serving system processes the prompt in chunks of T tokens (Agrawal et al., 2024), so each MoE layer sees $8T$ token-to-expert assignments whose distribution over experts is known only after its router runs. GPU systems absorb this variation with kernels that accept data-dependent shapes: block-sparse GEMMs that never drop a token (Gale et al., 2023), parallelism and pipelining chosen at run time (Hwang et al., 2023), and all-to-all exchanges sized per batch (Rajbhandari et al., 2022).

Server inference accelerators take a different path. Qualcomm Cloud AI 100 (Chatha, 2021), like XLA-compiled TPU programs (Lepikhin et al., 2021) and the statically scheduled Groq TSP (Abts et al., 2020), runs a model as a program compiled ahead of time: tensor shapes, the partition of every operator over cores and devices, and the schedule are fixed before the first request, and on Cloud AI 100 the compiler rejects control flow that depends on data. Static planning is what lets the compiler orchestrate 16 cores per SoC and four SoCs per card, but for MoE it turns two run-time quantities into compile-time constants: how many token rows each expert computes, and how expert outputs flow back to their tokens. Both grow costly at long chunks, which prefill uses to amortize weight traffic, and they have two structural consequences.

(i) Padding sized by the worst chunk. Each expert occupies a lane of fixed capacity, and every lane computes its full capacity however few tokens it receives. Routing is skewed and workload-dependent: in our routing captures, one expert in some layer receives 256 of 256 and 508 of 512 tokens, and the experts most loaded in one workload are rarely the most loaded in another (§3.1). A program that never drops a token therefore pads all 128 experts to T , $16\times$ the routed assignments. Calibrated per-expert capacities do not escape this: fitted to 200 prompts from all eight

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

workloads, they still overflow on 12–36% of each workload’s held-out prompts, and they converge on T .

(ii) Reductions shaped by the compiler rather than by the tokens. By default the partitioner slices each expert’s down projection across the four SoCs, which costs 121.6 MiB of cross-SoC traffic per layer, and it returns expert outputs to tokens through a dense accumulator per stage that is zero-filled, updated between stages, reduced in serialized steps and summed across SoCs on a single SoC, spilling to DRAM from $T = 256$ on. Even with the slicing removed and the worst serialization rewritten, this combine path takes 51%, 64% and 68% of an MoE layer at $T = 128, 256$ and 512 in the production MXFP6 precision (§3.2). Expert-parallel compilation alone makes the full model 46% faster at $T = 128$ but 11–12.5% slower than production at 256 and 512 tokens, because the dense combine grows with T .

We present RankTier, a design for MoE prefill that keeps the program static and moves the decisions the compiler cannot make into data. It rests on one property of the compiler: a program cannot branch on data, but it can address by data. A weight gather indexed by a run-time tensor is lowered to indirect weight DMA and runs as fast as a static weight stream (§3.3). RankTier therefore fixes lanes and capacities at compile time and decides at run time which expert each lane serves. Each SoC counts the tokens routed to its 32 experts, sorts them, and gives the expert of rank r a lane in the tier that covers r . Capacities bound to ranks can be small because the order statistics of per-SoC load are stable across workloads even when expert identities are not: over 200 held-out prompts of eight workloads, the 16th most loaded expert of a SoC never received more than 12 of 128 tokens, while a fixed expert order needs up to 84–127 rows at *every* position (Figure 1). Three tiers of 8, 8 and 16 lanes per SoC fit the compiler’s even core mapping and its sequential stages. For the reduction, RankTier compiles every expert whole on one SoC, restores the on-device KV cache that this setting breaks by making attention head-parallel, and lets each SoC own its tokens: it gathers only the rows of each token’s experts that live on the SoC, sums them locally, and leaves one elementwise cross-SoC sum that the compiler distributes as a reduce-scatter.

We implement RankTier as graph transformations of the exported model and evaluate the full 48-layer Qwen3-30B-A3B with MXFP6 expert weights on the four SoCs of one Cloud AI 100 Ultra card, with the KV cache retained on device. RankTier prefills 128-, 256- and 512-token chunks in 122, 206 and 356 ms, $4.05\times$, $2.45\times$ and $2.94\times$ faster than the production compilation. An ablation against a naive baseline that keeps the compiler’s combine and capacity T shows how the two parts interact: the reduction redesign alone gives 2.06 – $2.38\times$, the padding series alone at most

$1.05\times$, and the padding series on top of the reduction redesign another 1.06 – $1.37\times$. No lane exceeded its capacity in any measured prompt, and the error against an FP32 reference stays at the level of the production program.

The paper makes three contributions:

- A characterization of MoE prefill on an AOT-compiled multi-SoC accelerator, which identifies compile-time capacity and compiler-shaped reductions as the two costs that grow with the chunk, and establishes which run-time mechanisms the compiler does and does not offer (§3).
- RankTier: rank-bound tiered capacities built from run-time sorting and index-gathered weights, and a token-owned reduction on expert-parallel compilation with an on-device KV cache, all expressed as static graphs that the compiler maps evenly (§4–5).
- An end-to-end evaluation on the full model at three chunk lengths: a step-by-step ladder, a 2×2 ablation of the two optimization series, and the negative results that bound the design space (§6).

2 BACKGROUND AND RELATED WORK

2.1 MoE Inference

An MoE layer replaces the feed-forward block with E experts and a router. For a token with hidden state $x_t \in \mathbb{R}^d$, the router selects the set $\mathcal{K}_t$ of k experts with the largest gate values and returns

$$y_t = \sum_{e \in \mathcal{K}_t} w_{t,e} W_e^{\text{down}} (\text{SiLU}(W_e^{\text{gate}} x_t) \odot W_e^{\text{up}} x_t), \quad (1)$$

with normalized gate weights $w_{t,e}$. Qwen3-30B-A3B uses $E = 128$, $k = 8$, $d = 2048$ and an expert width of 768, so an expert’s three matrices hold 9 MiB in FP16. Implementations group assignments by expert, run each expert’s GEMMs over its gathered token rows, and return the weighted rows to their tokens; under expert parallelism the experts are spread over devices and dispatch and combine become cross-device exchanges (Lepikhin et al., 2021; Rajbhandari et al., 2022). Training systems bound each expert’s work by a capacity factor and drop the tokens beyond it (Lepikhin et al., 2021; Fedus et al., 2022); at inference, a dropped assignment changes the model’s output. Prefill processes the prompt in chunks of T tokens and writes their keys and values to the KV cache; decode then generates one token per step (Kwon et al., 2023; Agrawal et al., 2024). A chunk gives each layer $8T$ assignments, $2T$ per SoC on average when four SoCs hold 32 experts each.

2.2 MoE Systems

GPU systems handle the data-dependent shape of expert work directly. MegaBlocks expresses expert computation as block-sparse GEMMs and never drops tokens (Gale et al., 2023); Tutel switches parallelism and pipelining at run time (Hwang et al., 2023); DeepSpeed-MoE and FasterMoE optimize expert-parallel inference and training around the all-to-all exchange (Rajbhandari et al., 2022; He et al., 2022); Lina schedules all-to-all traffic and expert resources dynamically (Li et al., 2023). Placement studies exploit routing structure: ExFlow uses inter-layer expert affinity to cut all-to-all traffic (Yao et al., 2024), and Yu et al. (2025) profile data movement across workloads to derive a prefill-aware expert placement. Huang et al. (2024) characterize MoE inference and replace static capacity with dynamic gating; Pre-gated MoE predicts the next layer’s experts to overlap their loading (Hwang et al., 2024). Expert-choice routing gives every expert a fixed number of tokens by construction (Zhou et al., 2022), which removes the padding problem but changes the model. Closest to our setting, DynaMoE recompiles a static-shape program when capacities change during training (Kossmann et al., 2022). On our accelerator a full-model program takes 6–23 minutes to compile, so capacities cannot follow each chunk by recompilation, and none of the GPU mechanisms above exists in a program without data-dependent shapes.

2.3 AOT-Compiled Inference Accelerators

A Cloud AI 100 SoC has 16 AI cores and four 64-bit LPDDR4x channels with 136 GB/s and up to 32 GB (Chatha, 2021; Qualcomm Technologies, 2025). Each core combines a scalar VLIW processor, a vector unit and a tensor unit with 4K FP16 MACs, and holds an 8 MB software-managed vector memory and a 1 MB L2 cache. The Ultra card used here carries four SoCs, 64 cores in total, under a shared 150 W board limit.

The compiler takes an ONNX graph and one shape specialization per chunk length and produces a static program. It partitions each operator over the 16 cores of a SoC and, by multi-device tensor slicing, over the SoCs; the degree of weight splitting across SoCs is a compile option. Matrix weights can be stored as MXFP6 (Rouhani et al., 2023), which the vector unit dequantizes while the tensor unit computes. A KV cache stays on device when the compiler can pair each cache input with its updated output (*retained state*); otherwise the cache becomes host I/O. Control flow must be static: an ONNX `if` with a non-constant condition is rejected at compile time. Dynamic-shape compilers for CPUs and GPUs handle such workloads with shape functions and run-time dispatch (Shen et al., 2021; Zhu et al., 2021); the accelerator programs considered here have no such runtime.

3 MOTIVATION

We study the MoE layers of Qwen3-30B-A3B compiled for the four SoCs of one Cloud AI 100 Ultra card (setup in §6.1). The exported graph (Qualcomm Innovation Center, 2024) runs a layer’s 128 experts as two stages of 64 lanes; a lane is one expert’s gate, up and down GEMMs over a fixed number of gathered token rows, its capacity C . The production program uses $C = T$ in both stages, the only capacity that can never drop a token. Three observations motivate RankTier: capacities attached to experts must cover the worst chunk, the compiler’s combine grows with the chunk, and the compiler nevertheless lets a program address weights by data.

3.1 Capacities Attached to Experts Cover the Worst Chunk

Without data-dependent control flow a lane cannot skip rows: every lane of a compiled stage computes its full capacity of C rows, however few tokens it receives. Capacity is work, and the question is which capacities a program may use without dropping tokens.

We captured the routing of all 48 layers for 50 prompts of 128 tokens from each of eight workloads, GSM8K, MMLU, SWE-bench Lite issues, HumanEval and its JavaScript version, Alpaca instructions, CNN/DailyMail articles and Chinese XNLI sentence pairs (Cobbe et al., 2021; Hendrycks et al., 2021; Jimenez et al., 2024; Chen et al., 2021; Muenighoff et al., 2024; Taori et al., 2023; Hermann et al., 2015; Conneau et al., 2018), and for chunks of 256 and 512 tokens from four of them (Appendix B), with the model’s FP32 layers on the CPU. The load is skewed: in some layer one expert receives 127 of 128, 256 of 256 and 508 of 512 tokens. The skew is also workload-specific: across workloads the 16 most loaded experts of a layer agree with a Jaccard index of 0.00–0.33, against 0.52–0.88 between the two halves of one workload, and the 64 most loaded agree at 0.29–0.70 (HumanEval and its JavaScript version, which share their problem statements, are the exception).

A program that attaches capacities to experts, fixing which lane serves which expert, must reserve at each lane the largest load that its expert can receive. Figure 1 shows the cost. Ordering each SoC’s 32 experts by their mean load on calibration prompts and reading the held-out loads at each position of that order, the maximum is 84–127 of 128 rows at every position at $T = 128$: a drop-free fixed layout needs 91% of the rows of padding everything to T (73% at $T = 512$). Fitting stage capacities to calibration data does not rescue a fixed layout: with hot and cold stages fitted to 200 prompts from all eight workloads, 12–36% of each workload’s held-out prompts still overflow in some layer, and the fitted capacities approach the chunk length, 122 and 105 of 128 rows in the median layer.

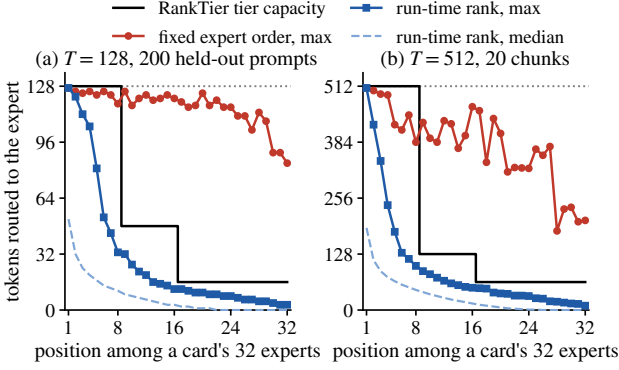


Figure 1. Rows a SoC must reserve for its expert at each position, as the largest load observed there. Red: experts in a fixed order, ranked by their mean load on calibration prompts (a: the first half of each workload’s prompts, evaluated on the other half; b: leave-one-out over the chunks). Blue: experts ordered at run time by their load in the chunk itself. Black: the tier capacities RankTier uses. Dotted: capacity T .

Padding is not free once T grows. In a one-layer replay, the tensor-unit time of a SoC’s 16 most loaded experts is 44, 310 and 921 μ s per core at capacities 92, 158 and 296, a $21\times$ increase for $3.2\times$ the rows, while their weight stream stays constant. In an instrumented profile of the full model at $T = 512$, the stage that holds each SoC’s 16 most loaded experts at capacity T takes a quarter of every layer (2.35 of 9.24 ms), and even after the reduction redesign of §4.2, capacity- T padding makes latency grow in proportion to the chunk (131.7, 258.9 and 498.4 ms at $T = 128, 256$ and 512; Table 3), so a longer chunk gains nothing per token.

3.2 The Compiler’s Combine Grows with the Chunk

The combine of Eq. (1), expert rows back to their tokens, is where the compiler’s defaults cost most.

Slicing across SoCs. With its default heuristic the partitioner splits the down-projection weights of all 64 lanes four ways along the output dimension, so each SoC computes a 512-column slice of every expert’s output. This requires an all-gather of the intermediate activation (36 MiB per stage at $C = 128$) and a re-shard of the output (24 MiB per stage) before the combine; cross-SoC traffic reaches 121.6 MiB per layer, while gate and up projections stay expert-parallel.

A dense accumulator. The exported combine scatters each stage’s weighted expert rows into an accumulator of shape $[64, T, d]$, zero-filled by a single core, gathered and updated between the two stages, reduced over each SoC’s 16 lanes by one einsum that the compiler runs through DRAM on one core, and finally summed over the four SoCs on four cores of SoC 0. From $T = 256$ on, this last sum also spills into a DRAM-backed reduction on one core (0.84 ms). Ev-

Table 1. Device time of one MoE layer with the dense combine, split into the combine path (the accumulator’s gap between the stages and SoC 0’s reduction tail) and everything else. MXFP6, expert-parallel compilation and the dense-path rewrites of §4.2, one-layer replays with stage capacities fitted to the input, instrumented.

T	Layer, ms	Combine path, ms	Rest, ms	Share
128	2.34	1.19	1.15	51%
256	4.35	2.78	1.57	64%
512	8.62	5.84	2.78	68%

ery part scales with T ; none scales with the number of routed rows.

Table 1 measures the combine path after expert-parallel compilation and after the rewrites of §4.2, which remove its most serialized parts: it still takes 51% of an MoE layer at $T = 128$ and 68% at 512. On the full model, compiling experts whole on each SoC with the KV cache kept on device (§4.1) reduces prefill latency from 494.6 to 267.1 ms at $T = 128$ but increases it from 505.3 to 568.6 ms at 256 and from 1049.1 to 1164.1 ms at 512 (Table 3), because its dense combine grows with T .

3.3 A Static Program Can Address by Data

What the compiler withholds is control flow; what it offers is indexing. We probed three mechanisms on one-layer replays and two-layer cuts of the full model.

Index-gathered weights are free. Replacing the three weight constants of a layer’s first stage by Gather (bank, idx), with the 64-entry index arriving at run time, leaves the layer at 2.796 ms, the same as with static weights, and the output is bit-identical: the compiler lowers the gather to an indirect weight DMA that reads the MXFP6 bytes directly. Only this form stays fast. Per-SoC constants and batched gathers run $2.3\text{--}3.7\times$ slower, because the compiler either keeps their weights in FP16 or spreads per-SoC MatMuls over all SoCs, and the one gathered constant is replicated on every SoC. A configuration with one partition per SoC stores each bank once, but the runtime then executes the SoCs one after another, $1.7\times$ slower for a single inference.

Lane counts must divide 16. The compiler maps a stage onto a SoC’s cores by its number of lanes per SoC. With a lane count that divides 16 (we tested 2, 4, 8 and 16) the mapping is balanced; with 17–19 lanes it splits the GEMMs unevenly (3 to 36 tensor-unit operations per core), and from 20 lanes on it packs the whole stage onto four cores per SoC, which takes 3.7–4.3 ms for a stage that runs in 0.5 ms on 16 cores.

Stages run one after another. The compiler schedules a

layer’s expert stages sequentially, in its own order; in our tiered programs a stage of L lanes per SoC ran one lane per core on L cores. Every additional stage adds a window of its own.

These properties define the space RankTier works in: fixed lanes whose experts are chosen by index at run time, tier widths from $\{1, 2, 4, 8, 16\}$ lanes per SoC, and as few stages as the padding allows.

4 RANKTIER

Section 3 leads to three requirements. First, capacity must follow each chunk’s load without control flow, so the program must choose at run time which expert a lane serves and bind capacities to a quantity that is stable across chunks. Second, the reduction must scale with the routed rows rather than with lanes $\times T$, and no SoC may root it. Third, every mechanism must be a static graph that the compiler maps evenly: lane counts from $\{1, 2, 4, 8, 16\}$ per SoC, one gathered constant per weight, and few stages.

Figure 2 shows one MoE layer under RankTier. §4.1 describes the compilation setting that makes experts local, §4.2 the reduction, §4.3 the run-time ranking and §4.4 the tiers.

4.1 Expert-Parallel Compilation with an On-Device KV Cache

We compile with a weight-splitting degree of one across SoCs (`-mdts-mos=1`), which forbids cutting any weight matrix across SoCs. The lane axis is then the only way to distribute the batched expert GEMMs, and each SoC computes 16 whole experts per stage, one per core; the all-gather and re-shard of the down projection disappear, and the layer’s cross-SoC traffic falls from 121.6 to 1.5 MiB.

The setting is global and also stops the compiler from splitting the attention projections by KV head. The cache update then comes out token-sliced while its input is whole, the compiler cannot pair the 96 cache inputs with their outputs, and all 192 KV buffers become host I/O. We restore the pairing at the graph level with *head-parallel attention*: the four KV-head groups of eight query heads each (Ainslie et al., 2023) become a leading batch axis of every attention operator, with per-group weights of shapes $[4, 2048, 1024]$ for queries, $[4, 2048, 128]$ for keys and values and $[4, 1024, 2048]$ for the output projection, whose four partial outputs are summed at the end. Under expert-parallel compilation the batch axis lands one group per SoC, the cache operations of KV head h stay on SoC h , and the program again exposes no KV buffer. On a two-layer cut the head-parallel program runs in 12.5 ms of device time against 23.3 ms for the production setting, with 30 instead of 265 MiB of cross-SoC traffic. The regrouped

Algorithm 1 Token-owned combine on SoC c

Require: top- k experts $\mathcal{K}[t, 1..k]$ of each token t ; lane map $\ell(e) \mapsto (s, c', l)$ (stage or tier, SoC, lane); slot $[e, t]$, row of token t in expert e ’s input; weighted outputs $O_s \in \mathbb{R}^{L_s C_s \times d}$ of SoC c

- 1: **for** all $t \leq T, j \leq k$ **in parallel** **do**
- 2: $e \leftarrow \mathcal{K}[t, j]$; $(s, c', l) \leftarrow \ell(e)$
- 3: **if** $c' = c$ **then**
- 4: $idx_s[t, j] \leftarrow l C_s + \text{slot}[e, t]$; $m_s[t, j] \leftarrow 1$
- 5: **else**
- 6: $idx_s[t, j] \leftarrow 0$; $m_s[t, j] \leftarrow 0$ {masked}
- 7: **end if**
- 8: **end for**
- 9: **for** each core q , owning tokens $t \in \text{tile } q$ **do**
- 10: $P_c[t] \leftarrow \sum_s \sum_j m_s[t, j] O_s[idx_s[t, j]]$ { $\leq k$ rows}
- 11: **end for**
- 12: $y \leftarrow (P_0 + P_1) + (P_2 + P_3)$ {elementwise, over SoCs}

output projection changes the FP16 summation order; on two layers the logits differ from the production program by 6.8×10^{-4} relative L_2 , with the same argmax.

4.2 Token-Owned Reduction

The reduction is rebuilt in three steps. Each keeps the model’s arithmetic and changes only where and in which order the sums run.

Dense-path rewrites. Three rewrites remove serialized work that the export introduces. (i) The first stage’s outputs were added to rows gathered from the freshly zeroed accumulator; we drop this read of zeros. (ii) The per-SoC sum over lanes, one einsum that the compiler ran through DRAM on one core, is cut into 16 tiles of $T/16$ tokens. (iii) The routing’s prefix sums over tokens, which give each assignment its row in the expert’s compact input and ran as a serial CumSum on one core, become Hillis–Steele scans of $\log_2 T$ shift-and-add steps (Hillis & Steele, 1986). All three leave the output bit-identical.

Token-owned combine. The dense accumulator exists because the export combines by lane, scattering each lane’s rows to their tokens. RankTier combines by token (Algorithm 1). The router’s top- k output already lists each token’s experts; with integer arithmetic on the lane layout and the per-expert slot tables that the routing computes anyway, every (token, j) pair maps to the stage and row of the SoC’s compact expert outputs that hold its result, or to a masked sentinel when the expert lives on another SoC. One gather per stage then fetches at most eight rows per token; masking, the addition of the stages and a sum over the rows give the SoC’s $[T, d]$ partial output. Each core owns a tile of $T/16$ tokens, so the per-SoC combine needs no cross-core traffic.

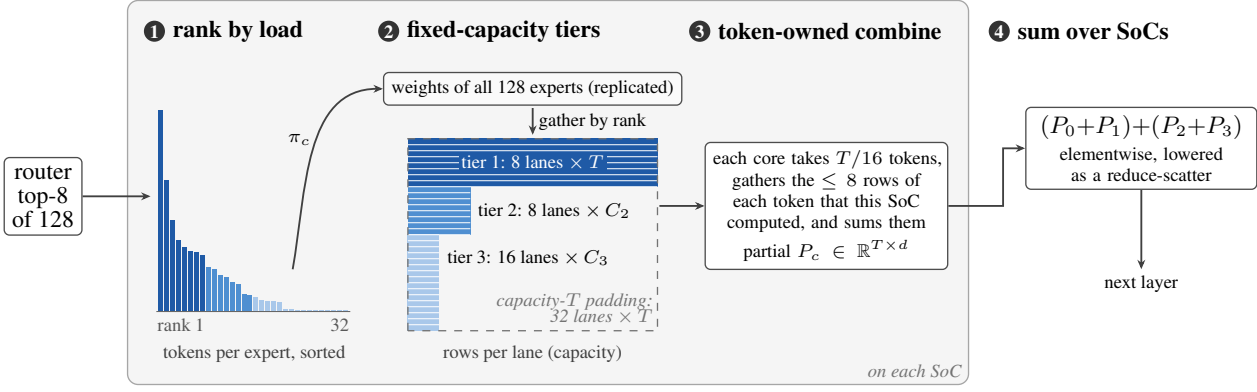


Figure 2. One MoE layer under RankTier. (1) Each SoC counts the tokens routed to its 32 experts and sorts them (bars: SoC 2 in layer 20 of the held-out 512-token chunk; colors mark the tiers). (2) Three tiers of lanes with compile-time capacities ($C_2 = 128$, $C_3 = 64$ at $T = 512$) serve the experts by rank; their weights are gathered from a bank of all 128 experts through the rank permutation π_c . The filled area is the padded work, against the dashed capacity- T layout. (3) Each core gathers and sums the rows of its own tokens. (4) One elementwise sum over SoCs completes the layer.

Two lowering details matter. The device gather does not zero-fill out-of-range indices, so the safe index and the explicit mask are both required. And the per-SoC work must stay on the lane axis that the compiler already splits over SoCs: written as explicit per-SoC slices, the combine made the compiler distribute its tiles over SoCs and ship expert outputs between them, raising cross-SoC traffic from 3 to 78 MiB and the layer’s device time from 4.3 to 8.6 ms.

Elementwise final sum. The remaining sum over the four partials was written as 16 einsum tiles, which the compiler lowers to a reduction template on four cores per SoC. Written as the elementwise tree $(P_0 + P_1) + (P_2 + P_3)$ per token tile, the additions spread over all 16 cores of each SoC. In the full model the compiler lowers this sum as a reduce-scatter: each SoC adds a quarter of the tokens, and the next layer’s input normalization, which runs split by token rows, gathers the rest. No SoC receives all four partials.

4.3 Run-Time Rank Sort

Each SoC owns a fixed quarter of the 128 experts. After the router, RankTier counts each expert’s tokens from the routing mask and sorts each SoC’s 32 counts (a `TopK`), which gives a per-SoC permutation π_c from rank to expert. Lane j of the tier starting at rank a_i serves the expert of rank $a_i + j$; its three weight matrices come from `Gather(bank,  $\pi$ )` on one constant that holds all 128 experts, lowered to indirect DMA (§3.3).

Two choices keep the sort off the critical path. First, the routing mask, the prefix scans and the slot tables are computed once over all 128 experts in native order, independent of the sort; only a row permutation of these tables and the weight gathers wait for it. Sorting first and running the routing on permuted columns makes everything wait (12 layers: +2.5 ms over capacity- T padding, against +1.6 ms). Sec-

ond, gathered weights are sensitive to the compiler’s maximum tile size: with the default, the first expert GEMM waits 0.085 ms for its first gathered tile, with 512 KiB tiles 0.032 ms. The best size is 512 KiB for $T \leq 256$ and 1024 KiB for $T = 512$ (§6.5).

The price is memory. The gathered constant is replicated, so every SoC holds the weights of all 128 experts, 22.5 GiB per SoC instead of 6.7 GiB, within the SoC’s 32 GB. The program also returns every lane’s token count in every layer, so an overflow, a rank that received more tokens than its tier’s capacity, is detectable after each chunk.

4.4 Tiered Capacities

A tier is a stage of L_i lanes per SoC at capacity C_i that covers ranks $a_i, \dots, a_i + L_i - 1$, with $\sum_i L_i = 32$. Every expert keeps exactly one lane, so tiers change only the padding: the same experts compute the same tokens, and the output is bit-identical to the two-tier sort. A SoC computes $\sum_i L_i C_i$ rows per layer, against $32T$ with capacity- T padding.

Capacities from order statistics. The first tier has capacity T , since rank 1 can receive every token. For $i > 1$, C_i is the largest load observed at rank a_i , the tier’s first rank, over calibration chunks, times a headroom $h = 1.28$, rounded up to a multiple of 16 and capped at T ; since ranks are sorted, rank a_i bounds the whole tier. We calibrate on routing captures of the eight workloads: the 128-token prompts of §3.1, 56 chunks of 256 tokens and 20 of 512. Order statistics are predictable for a structural reason: if a SoC receives n_c assignments in a chunk, its r -th most loaded expert holds at most n_c/r of them, and n_c is a sum over 32 experts, $2T$ on average and at most $1.9\times$ that in our 256- and 512-token captures. An expert-bound capacity must instead cover the worst case of one expert, which is T .

Table 2. Tier designs, lanes per SoC $\times$ capacity, and the padded rows each SoC computes per layer.

T	Ranks 1–8	9–16	17–32	Rows	vs. two tiers / T/T
128	8×128	8×48	16×16	$13T$	72% / 41%
256	8×256	8×64	16×32	$12T$	67% / 38%
512	8×512	8×128	16×64	$12T$	67% / 38%

Choosing the tiers. Rows alone favor many tiers: under our calibration three stages need 65–67% of the rows of a two-tier split at T and $T/8$, four stages 52–58% and five 48–56%. But stages run sequentially and each adds a window of its own, so the design balances rows against stages. At $T = 512$ the first tier is bound by the SoC’s shared memory traffic rather than by per-core compute: eight lanes of 512 rows finish in 0.8–1.0 ms, sixteen lanes in 2.2 ms, although each core does the same work. RankTier uses three tiers of 8, 8 and 16 lanes per SoC (Table 2), $12\text{--}13T$ rows per SoC against $18T$ for two tiers and $32T$ for capacity- T padding; §6.5 shows that a fourth tier costs more than it saves.

5 IMPLEMENTATION

RankTier is a set of ONNX graph transformations, about 1,100 lines of Python, applied to the Qwen3-30B-A3B graph that the vendor’s QEfficient library exports for Cloud AI 100 (Qualcomm Innovation Center, 2024). Each step of §4 is a separate pass, so every intermediate program can be compiled and measured. The passes locate operators by structure rather than by name (layer 0 numbers some nodes differently and folds a 64-lane constant into its stage), regenerate the T -dependent constants of the scans and tiles for each chunk length, and clone the stage subgraph once per tier with its own row order, weight gathers and capacity. A static `If` in the export that squeezes the token table is replaced by the equivalent `Squeeze` in every clone, and the combine’s map from a lane to (tier, SoC, row) becomes a set of small per-tier tables. For the ablation (§6.3) the same pass can instead feed the export’s own combine.

Each chunk length is its own program, with sequence length T and context $2T$. We compile with MXFP6 matrix weights, FP16 activations, 16 cores per SoC, depth-first scheduling, expert-parallel slicing and retained state (Appendix A). Static programs compile in 6–12 minutes and occupy 25 GB; programs with gathered weights compile in 12–23 minutes and occupy 88 GB, because the program file keeps an FP16 copy of each gathered bank.

6 EVALUATION

We ask how much RankTier speeds up the full model (§6.2), what each part of the reduction and of the padding design

contributes (§6.4, §6.5), whether the output is preserved (§6.6), and which alternatives failed (§6.7).

6.1 Experimental Setup

Hardware and model. One Cloud AI 100 Ultra card, four SoCs with 16 cores and 32 GB each, platform SDK 1.21.6. Qwen3-30B-A3B, all 48 layers, batch 1, MXFP6 matrix weights and FP16 activations as in the production deployment. Every program keeps its KV cache on device.

Programs. Seven programs, each adding one step to the one before it (Table 3): (1) the production compilation of the exported graph, with default partitioning, the dense combine and capacity T in both stages; (2) expert-parallel compilation with head-parallel attention, which keeps the export’s combine and capacity T and is our *naive* T/T baseline; (3) the dense-path rewrites; (4) the token-owned combine; (5) the elementwise final sum; (6) the run-time rank sort with two tiers at T and $T/8$; (7) RankTier’s three tiers.

Prompts. At $T = 128$ the first chunk of a GSM8K chat prompt; at 256 and 512 held-out chunks built from GSM8K test questions outside the calibration range. Latency depends on a chunk only through its routing, and on a single layer it varied by about 1% across 200 prompts of four workloads.

Protocol. Host latency of one chunk, the median of three rounds of 20 inferences after warm-up. All programs of one chunk length run back to back in one session, with anchor programs repeated before and after; steps 6 and 7 ran in a second session and are chained to the first through step 5, which ran in both and agreed within 0.13%. Rounds agree within 0.6%. For accuracy we compare the last-position logits with an FP32 CPU run of the same chunk, check the next-token prediction, and read back every lane’s token count.

6.2 End-to-End Results

Table 3 and Figure 3 give the results. RankTier prefills 128-, 256- and 512-token chunks in 122.2, 206.2 and 356.3 ms, $4.05\times$, $2.45\times$ and $2.94\times$ faster than the production compilation, and $2.19\times$, $2.76\times$ and $3.27\times$ faster than naive T/T . In the order of the ladder, the reduction steps contribute $2.06\text{--}2.37\times$ and the padding steps after them $1.06\text{--}1.38\times$, both growing with T ; §6.3 separates the two series. Only with both does a longer chunk pay: per-token latency falls from 0.95 to 0.70 ms between $T = 128$ and 512 under RankTier, stays at 0.97–1.03 ms with the token-owned combine at capacity T , and rises from 2.09 to 2.27 ms under naive T/T .

Table 3. Prefill latency of one chunk (ms) for the seven programs, and the change against the program above it.

Program	$T = 128$	$T = 256$	$T = 512$
1 Production	494.6	505.3	1049.1
2 + expert/head par.	267.1 −46.0%	568.6 +12.5%	1164.1 +11.0%
3 + rewrites	158.4 −40.7%	355.9 −37.4%	729.1 −37.4%
4 + token-owned	131.7 −16.9%	258.9 −27.3%	498.4 −31.6%
5 + elementwise sum	129.5 −1.6%	252.1 −2.6%	491.1 −1.5%
6 + rank sort	126.3 −2.5%	215.2 −14.6%	421.6 −14.2%
7 + tiers (RankTier)	122.2 −3.2%	206.2 −4.2%	356.3 −15.5%
Reductions, 2→5	2.06×	2.26×	2.37×
Padding, 5→7	1.06×	1.22×	1.38×
vs. naive T/T , 2→7	2.19×	2.76×	3.27×
vs. production	4.05×	2.45×	2.94×
Tokens/s, 1 / 7	259 / 1047	507 / 1241	488 / 1437

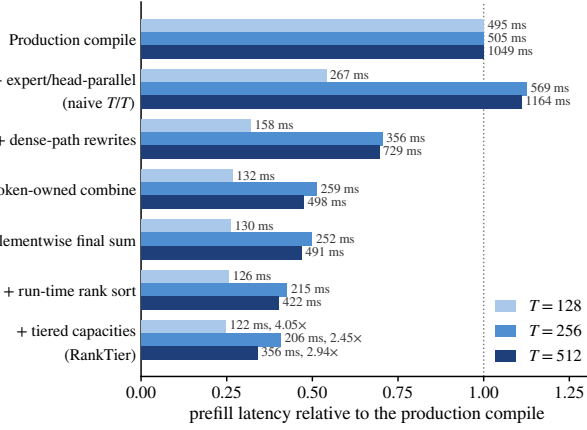


Figure 3. Prefill latency of the seven programs relative to the production compilation at the same chunk length, with the latency of each program and the speedup of RankTier.

6.3 Ablation of the Two Series

The ladder adds the reduction steps before the padding steps, so it cannot show what each series gives alone. Table 4 starts both from naive T/T and adds programs that apply only the padding series: the rank sort and the tiers feeding the export’s own combine. The export’s scatter writes only into an accumulator with as many lanes as the stage, so tiers of equal width share one accumulator, read-modify-written as the export’s second stage is, and tiers of 8, 8 and 16 lanes per SoC need two accumulators, of 32 and 64 lanes, where the two-tier sort needs one.

The reduction series alone gives 2.06–2.38×. The padding series alone gives almost nothing: the rank sort 1.00–1.05×, and with the three tiers the program runs at only 0.69–0.85× the baseline’s speed. The same padding series gives 1.06–1.37× once the reduction series is in place. Two effects explain the difference. First, padding shortens only the expert stages, while most of the naive layer is combine work that grows with lanes × T (Table 1): the 2.8 ms

Table 4. The two series alone and together on the full model: prefill latency per chunk (ms) and speedups over naive T/T (N). One session per chunk length, each program timed twice in mirrored order; P_2 ran in a second session, chained through N and P.

	Reduction	Padding	$T = 128$	$T = 256$	$T = 512$
N	export	T/T	267.3	569.9	1162.2
R	ours	T/T	129.6	253.2	488.7
P ₂	export	rank sort	267.4	551.3	1110.3
P	export	sort, tiers	388.4	669.7	1379.3
D	ours	sort, tiers	122.6	206.8	355.8
Reduction alone, N/R			2.06×	2.25×	2.38×
Rank sort alone, N/ P_2			1.00×	1.03×	1.05×
Sort and tiers alone, N/P			0.69×	0.85×	0.84×
Padding after the reduction, R/D			1.06×	1.22×	1.37×
Both (RankTier), N/D			2.18×	2.76×	3.27×

per layer that padding saves at $T = 512$ is 27% of the reduced layer, and the same saving would be only 11% of the 24.2 ms naive layer. Second, under the export’s combine the tiers cost accumulators: the second accumulator adds a zero fill and a lane reduction through DRAM, and the third tier costs 2.5, 2.5 and 5.6 ms per layer at $T = 128$, 256 and 512 (not profiled). In the token-owned combine, a tier adds only one gather of at most eight rows per token. The reduction redesign is therefore a prerequisite for the padding series, not an independent gain.

6.4 Reduction

Expert parallelism alone is not enough. Step 2 removes the down projection’s slicing and is 46% faster than production at $T = 128$. At 256 and 512 tokens it is 12.5% and 11.0% slower, because the dense combine that remains grows with T (Table 1); the head-parallel attention in this step only restores the on-device cache.

The rewrites and the token-owned combine carry the reduction. The dense-path rewrites save 37–41% at every chunk length and the token-owned combine another 16.9%, 27.3% and 31.6%, growing with T as the accumulator it removes does. The mechanism is visible on one layer in isolation: with the same fitted capacities, the token-owned layer runs in 2.19, 3.23 and 5.69 ms at $T = 128$, 256 and 512 against 2.61, 5.30 and 9.93 ms for the dense combine; at $T = 256$ the work after the last expert stage falls from 2.0 ms to under 0.1 ms per SoC plus the cross-SoC sum. The elementwise final sum adds 1.5–2.6%.

What remains is small. In a two-layer profile at $T = 512$, the MoE partial sums exchanged between SoCs are 6 MiB and 0.30–0.34 ms per layer, 14% of the cross-SoC bytes, and the tail from the last expert GEMM to the completed sum is 0.48 ms. Removing the exchange entirely would save about 3% of the model. For this reason we did not pursue placements that make tokens SoC-local (§6.7).

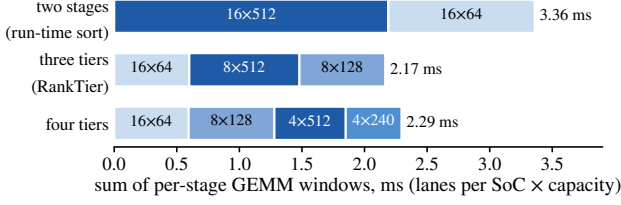


Figure 4. GEMM window of each expert stage in execution order at $T = 512$, two-layer profiles; segments are the per-stage medians over SoCs, labeled with lanes per SoC $\times$ capacity.

Table 5. Rank sort on 12-layer cuts: latency change against the token-owned program at capacity T (default tiles) for three maximum tile sizes.

T	Default tiles	512 KiB	1024 KiB
128	+4.7%	-1.4%	-
256	-5.1%	-18.2%	-7.9%
512	-12.7%	-9.3%	-15.9%

6.5 Padding

The rank sort pays from 256 tokens on. With two tiers at T and $T/8$, step 6 saves 2.5% at $T = 128$, where the expert stages stream weights at their bandwidth floor and the cold stage’s 112 fewer rows per lane hide under that stream, and 14.6% and 14.2% at 256 and 512. The sort adds a dependency on every layer’s critical path, router to sort to weight gathers; with the tile sizes below it costs less than it saves even at $T = 128$.

Three tiers, not more. Step 7 saves another 3.2%, 4.2% and 15.5%. Figure 4 shows why three tiers are the right number at $T = 512$. With two tiers, the 16 lanes at capacity 512 take 2.2 ms per layer; split into eight lanes at 512 and eight at 128, the same ranks take 0.8–1.0 and 0.65–0.72 ms, although each core of the eight-lane stage does the same work as before, because the stage is bound by the SoC’s memory traffic. A fourth tier removes more rows but adds a sequential window: on two layers, four tiers are 8.6% faster than two, three tiers 14.1%. At $T = 256$ the extra stage’s window absorbs most of the saving (Table 3).

Tile size. Table 5 shows the sensitivity of gathered weights to the compiler’s maximum tile size. With default tiles the rank sort is 4.7% slower than the token-owned program at capacity T at $T = 128$; with 512 KiB tiles it is 1.4% faster. The best size moves with the chunk, 512 KiB up to 256 tokens and 1024 KiB at 512; static weights prefer the default, which small tiles slow by 1–2%. Each program therefore uses its own best size.

Capacity usage. The largest lane loads observed in the timed chunks were 122, 20 and 8 of capacities 128, 48 and 16 at $T = 128$; 251, 36 and 17 of 256, 64 and 32 at 256; and 508, 83 and 37 of 512, 128 and 64 at 512. The first tier is

Table 6. Relative L_2 error of the last-position logits against FP32, and the next token ($\checkmark$: matches FP32).

Program	$T = 128$	$T = 256$	$T = 512$
1 Production	0.188 $\checkmark$	0.089 $\times$	0.095 $\checkmark$
2–3 Naive T/T , rewrites	0.172 $\checkmark$	0.077 $\checkmark$	0.094 $\checkmark$
4 Token-owned combine	0.172 $\checkmark$	0.073 $\checkmark$	0.092 $\checkmark$
5 Elementwise sum	0.190 $\checkmark$	0.077 $\checkmark$	0.095 $\checkmark$
6–7 Rank sort, RankTier	0.170 $\checkmark$	0.087 $\times$	0.088 $\checkmark$

Table 7. Alternatives we built or analyzed and rejected ($T = 512$ unless noted).

Alternative	Result	Reason
Sort first, then route on permuted columns	+2.5 vs. +1.6 ms (12 layers, $T=128$)	all routing waits for the sort
Row-chunked expert GEMMs	+25%, +66% (one layer)	each chunk re-streams and re-dequantizes the weights
One partition per SoC	1.7 $\times$ slower	the runtime runs partitions in sequence
Per-layer first-tier capacity	-10–15% of its rows	held-out chunks overflow in 1–5 layers
Co-locate experts that fire together	3.63 $\rightarrow$ 2.42 SoCs per token	doubles lower-tier loads; the exchange is 3%
Split first-tier experts over two lanes	+6% (two layers)	16 lanes stay on 8 cores; 1.8 $\times$ dequantization
Four tiers	-8.6% vs. -14.1%	one more sequential window
No depth-first scheduling	+1–2%	stages still run in sequence

nearly full in every case, which is why its capacity stays at T : taking each layer’s calibration maximum instead would remove 10–15% of its rows but overflow in one and five layers of the held-out chunks.

6.6 Accuracy

Every program stays at the error level of the production program’s MXFP6 weights (Table 6). The programs differ from production, and from one another, only in FP16 summation order: head-parallel attention regroups the output projection, which alone moves 343 of 49,152 routing assignments at $T = 128$, and the token-owned combine sums a token’s rows in a different order; neither leaves that level. Tiers are bit-identical to the two-tier sort, since the same experts compute the same tokens. The next token matches FP32 in every case except the 256-token chunk, whose two leading FP32 logits are 0.58 apart; production and the rank-sorted programs pick the second. No lane exceeded its capacity in any program.

6.7 What Did Not Work

Table 7 lists the alternatives we rejected. Two are instructive. Splitting a heavily loaded expert over several lanes, each with a chunk of its rows, halves the first tier’s GEMM time in isolation, but the compiler maps the resulting 16-lane stage onto the eight cores of the unsplit one and raises its dequantization $1.8\times$, so the program gets slower. Co-locating experts that fire together cuts the SoCs a token touches from 3.63 to 2.42 and puts 12% of the tokens on a single SoC, but it concentrates load, doubles the order statistics the lower tiers must cover (at $T = 512$ from 50 to 85 rows at rank 17), and targets an exchange that is only 3% of the model; spreading such experts over SoCs instead lowers those statistics.

6.8 Discussion and Limitations

Memory. Replicating every expert’s weights on every SoC costs 22.5 instead of 6.7 GiB per SoC, which limits the room for KV cache and batching. The replication is a property of the compiler’s fast path (§3.3), not of the design: each SoC reads only its own 32 experts, and a per-operator device placement would store each bank once.

Statistical capacities. Tiers 2 and 3 rely on order statistics calibrated with $1.28\times$ headroom. The held-out chunks fit, and the program reports every lane’s load, so an overflow is detectable after the chunk; a deployment could then re-run the chunk with a capacity- T program, which we have not implemented.

Scope. We evaluate one model at batch 1 and three chunk lengths, and only the first prefill chunk: correctness of a second chunk against a filled cache, and decode, are untested for the expert-parallel programs. End-to-end latencies come from one prompt per chunk length, which is representative because latency depends on the routing only weakly.

Power. The four SoCs share one 150 W limit. During a 512-token run the board peaks at 154–164 W, SoCs 0 and 3 drop from 1100 to 941 MHz, and their expert stages finish up to 0.5 ms later than those of the other SoCs; the effect follows the physical SoC and adds run-to-run variation, which is why we compare programs only within a session.

Beyond Cloud AI 100. RankTier needs two things from a static-shape stack: a sort, and a gather with run-time indices that the compiler lowers to efficient weight DMA. Other AOT stacks such as XLA offer both operators, so rank-bound capacities should carry over, but whether their gathers of weights are as cheap as here is untested.

What remains. After RankTier, the largest item of a 512-token layer is attention’s sum over its four head groups, which the compiler lowers to batched reductions on four

cores per SoC, partly through DRAM; it lies outside the MoE layer studied here.

7 CONCLUSION

Ahead-of-time compilation fixes two things that MoE prefill would rather decide per chunk: how many rows each expert computes, and how expert outputs return to their tokens. RankTier keeps the program static and moves both decisions into data. Capacities are bound to per-SoC load ranks, whose order statistics are stable across workloads when expert identities are not, and the experts that fill each rank are gathered by index at run time, which the compiler supports at no cost. The combine is organized by token ownership, so it scales with routed rows and needs no root SoC, and an ablation shows that the padding series pays only once this combine is in place. On Qwen3-30B-A3B across the four SoCs of a Cloud AI 100 Ultra card, RankTier prefills 128-, 256- and 512-token chunks $4.05\times$, $2.45\times$ and $2.94\times$ faster than the production compilation with the KV cache on device. The broader lesson for static-shape accelerators is that the order statistics of expert load are far more predictable than expert identities, which makes them the quantity to compile.

REFERENCES

- Abts, D., Ross, J., Sparling, J., Wong-VanHaren, M., Baker, M., Hawkins, T., et al. Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 145–158, 2020.
- Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Tam- ing throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 117–134, 2024.
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4895–4901, 2023.
- Chatha, K. Qualcomm cloud AI 100: 12 TOPS/W scalable, high performance and low latency deep learning inference accelerator. In *2021 IEEE Hot Chips 33 Symposium (HCS)*, pp. 1–19, 2021. doi: 10.1109/HCS52781.2021.9567417.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. Evaluating

- large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Conneau, A., Rinott, R., Lample, G., Williams, A., Bowman, S., Schwenk, H., and Stoyanov, V. XNLI: Evaluating cross-lingual sentence representations. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2475–2485, 2018.
- DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Gale, T., Narayanan, D., Young, C., and Zaharia, M. MegaBlocks: Efficient sparse training with mixture-of-experts. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.
- He, J., Zhai, J., Antunes, T., Wang, H., Luo, F., Shi, S., and Li, Q. FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pp. 120–134, 2022.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.
- Hermann, K. M., Kočíský, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., and Blunsom, P. Teaching machines to read and comprehend. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.
- Hillis, W. D. and Steele, Guy L., J. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.
- Huang, H., Ardalani, N., Sun, A., Ke, L., Lee, H.-H. S., Bhosale, S., Wu, C.-J., and Lee, B. Toward efficient inference for mixture of experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.
- Hwang, C., Cui, W., Xiong, Y., Yang, Z., Liu, Z., Hu, H., Wang, Z., Salas, R., Jose, J., Ram, P., Chau, J., Cheng, P., Yang, F., Yang, M., and Xiong, Y. Tutel: Adaptive mixture-of-experts at scale. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.
- Hwang, R., Wei, J., Cao, S., Hwang, C., Tang, X., Cao, T., and Yang, M. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 1018–1031, 2024.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Kossmann, F., Jia, Z., and Aiken, A. Optimizing mixture of experts using dynamic recompilations. *arXiv preprint arXiv:2205.01848*, 2022.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626, 2023.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.
- Li, J., Jiang, Y., Zhu, Y., Wang, C., and Xu, H. Accelerating distributed MoE training and inference with Lina. In *2023 USENIX Annual Technical Conference (USENIX ATC)*, pp. 945–959, 2023.
- Muennighoff, N., Liu, Q., Zebaze, A., Zheng, Q., Hui, B., Zhuo, T. Y., Singh, S., Tang, X., von Werra, L., and Longpre, S. OctoPack: Instruction tuning code large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Qualcomm Innovation Center. Efficient transformers library (QEfficient). <https://github.com/quic/efficient-transformers>, 2024.
- Qualcomm Technologies. Cloud AI 100 architecture. Qualcomm Cloud AI SDK documentation, <https://quic.github.io/cloud-ai-sdk-pages/latest/Getting-Started/Architecture/>, 2025.

- Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R. Y., Awan, A. A., Rasley, J., and He, Y. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pp. 18332–18346, 2022.
- Rouhani, B. D., Zhao, R., More, A., Hall, M., Khodamoradi, A., Deng, S., Choudhary, D., Cornea, M., Dellinger, E., Denolf, K., et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- Shen, H., Roesch, J., Chen, Z., Chen, W., Wu, Y., Li, M., Sharma, V., Tatlock, Z., and Wang, Y. Nimble: Efficiently compiling dynamic neural networks for model inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, pp. 208–222, 2021.
- Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., and Hashimoto, T. B. Stanford Alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Yang, A. et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yao, J., Anthony, Q., Shafi, A., Subramoni, H., and Panda, D. K. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. *arXiv preprint arXiv:2401.08383*, 2024.
- Yu, Z., Guan, Y., Yu, Z., Zhou, C., Hu, Z., Pei, S., Kang, Y., Ding, Y., and Tsai, P.-A. Patterns behind chaos: Forecasting data movement for efficient large-scale MoE LLM inference. *arXiv preprint arXiv:2510.05497*, 2025.
- Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A., Chen, Z., Le, Q., and Laudon, J. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- Zhu, K., Zhao, W., Zheng, Z., Guo, T., Zhao, P., Bai, J., Yang, J., Liu, X., Diao, L., and Lin, W. DISC: A dynamic shape compiler for machine learning workloads. In *Proceedings of the 1st Workshop on Machine Learning and Systems (EuroMLSys)*, pp. 89–95, 2021.

A COMPILATION FLAGS AND PROGRAM SIZES

All programs are compiled with the command below, with a four-SoC partition configuration and one shape specialization per chunk length (sequence length T , context $2T$). Steps 2–7 add `-mdts-mos=1`; steps 6–7 also set the maximum tile size, 512 KiB for $T \leq 256$ and 1024 KiB for $T = 512$.

```
gaic-compile -aic-hw -aic-hw-version=ai100
  -retained-state -convert-to-fp16
  -mxfp6-matmul -aic-num-cores=16 -mos=1
  -aic-enable-depth-first
  -mdp-load-partition-config=<4 SoCs>
  -network-specialization-config=<T, 2T>
  [-mdts-mos=1] # steps 2-7
  [-size-split-granularity=512|1024] # 6-7
```

Table 8 lists the program sizes.

Table 8. Program file, device memory per SoC and compile time.

Programs	File	Memory/SoC	Compile
Steps 1–5, static weights	25 GB	6.6–7.1 GiB	6–12 min
Steps 6–7, gathered	88 GB	22.4–22.5 GiB	12–23 min

B ROUTING CAPTURES AND TIER CALIBRATION

Routing captures run the model’s FP32 layers on the CPU and record every layer’s top-8 experts. At $T = 128$ we use 50 chat prompts per workload; at 512 tokens, 8 GSM8K chunks and 4 each from HumanEval, MMLU and SWE-bench Lite; the 256-token calibration set holds 16 GSM8K chunks and both halves of each 512-token chunk (56 in total). The held-out GSM8K chunks of the evaluation use test questions 1000–1003 (256 tokens) and 1004–1011 (512 tokens), outside the calibration range. Table 9 gives the largest per-SoC load at selected ranks, the input to the tier rule of §4.4.

Table 9. Largest load of the rank- r expert of a SoC over all calibration chunks, layers and SoCs.

T	$r = 1$	2	4	8	9	16	17	32
128	127	122	105	33	32	14	12	3
256	256	223	134	58	50	25	25	4
512	508	424	240	101	89	51	50	10

A fixed layout calibrated per workload fares worse the more its traffic varies. With 25 calibration prompts of the same workload, 64–92% of that workload’s held-out prompts overflow in some layer; calibrated on 175 prompts of the seven other workloads, 24–100% do. With 200 prompts

from all eight, the fraction falls to 12–36%, and with random pooled subsets of 8, 32, 128 and 200 prompts to 100%, 70%, 27% and 21%. Under the per-SoC rank sort, ranks 17–32 need at most 12 rows at $T = 128$ in every layer of every workload.

C ADDITIONAL ONE-LAYER MEASUREMENTS

On a one-layer replay (layer 2, MXFP6, expert-parallel), the combine and the capacities interact. With capacity T in both stages, a token-centric predecessor of the token-owned combine saves 12%, 29% and 23% at $T = 128, 256$ and 512; fitted capacities save 6%, 14% and 13% with the dense combine but 9%, 15% and 25% once the combine is fixed, since the padded compute is then exposed. Both together are 20%, 40% and 42% faster than the naive layer.